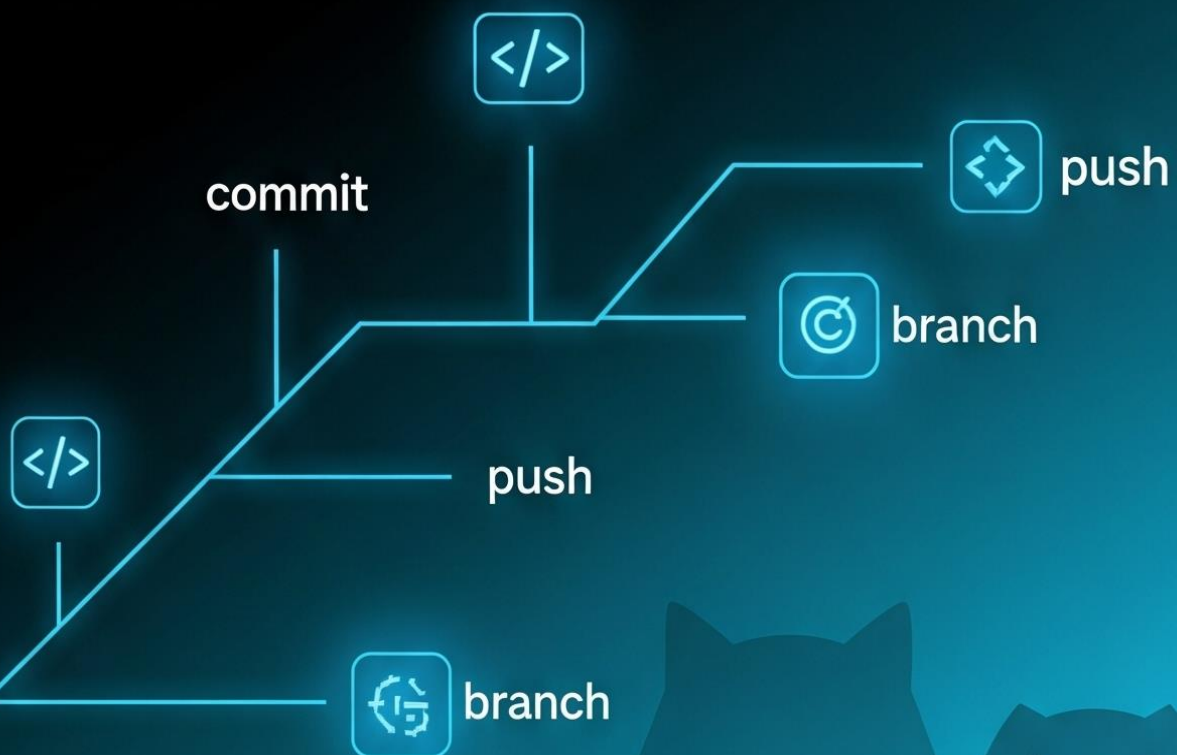


# Fundamentos para Versionamiento de Código



## O que é Branch no Git/GitHub?

Branch significa “**ramo**” (como galho de árvore).

No Git, uma branch é uma linha de desenvolvimento independente.

Cada branch é como uma versão paralela do seu projeto, onde você pode fazer mudanças sem mexer diretamente no "tronco principal".

### ✂ Analogia:

Imagine que seu projeto é uma árvore 🌳.

O tronco principal é a branch main.

Quando você quer testar algo novo sem estragar o que já funciona, você cria um galho (branch) separado.

Trabalha nesse galho, faz testes, e se der certo, você cola de volta no tronco (isso é o merge).

### ◆ O que é a Main no GitHub?

A main é a branch padrão do GitHub.

Antes de 2020, o nome padrão era master, mas foi alterado para main.

Normalmente, a main representa a versão oficial e estável do seu projeto.

### ✂ Analogia:

Pense em uma loja online:

O site que os clientes usam é a branch main (a versão oficial e estável).

Se você quiser adicionar um novo recurso, cria uma branch nova (feature-carrinho, por exemplo), trabalha nela e depois junta com a main.

### ◆ Estrutura básica

Cada branch guarda um histórico de commits.

Você pode ter várias branches ao mesmo tempo.

### Exemplo comum:

**main** → código estável (produção).

**dev** → desenvolvimento em andamento.

**feature-x** → recurso específico.

**bugfix-y** → correção de erro.

## ◆ Como funciona na prática (comandos básicos)

# Ver branch atual

**git branch**

# Criar uma nova branch

**git branch nova-feature**

# Mudar para essa branch

**git checkout nova-feature**

# Criar e já mudar

**git checkout -b nova-feature**

# Depois que terminar, unir com a main

**git checkout main**

**git merge nova-feature**

**Aqui vai um esquema em ASCII art para visualizar a ideia de branch, main e como eles se conectam:**

Estrutura simplificada de branches no Git

**(main - tronco principal)**

o---o---o---o---o

\

\

**(feature - um galho para desenvolver algo novo)**

o---o---o

## ✦ Explicação:

Os círculos “○” são commits (salvamentos da história do projeto).

A linha de cima é a branch main (tronco).

A linha de baixo é a branch feature (um galho novo saindo do tronco).

🌳 Com merge (quando juntamos o galho de volta no tronco)

(main - tronco principal)

o---o---o---o---o-----o

\                      /

\                      /

(feature - galho que foi unido) o---o

🔗 Explicação:

Você trabalhou na feature (galho).

Quando terminou, fez um merge → o galho volta para o tronco (main).

Agora a main tem as novidades, e o projeto segue com tudo junto.

👉 Dá para imaginar seu projeto realmente como uma árvore:

**Main** = tronco estável.

**Branches** = galhos que nascem do tronco para testar ou criar algo novo.

**Merge** = colar o galho de volta no tronco.

✅ Resumo da analogia:

**Projeto** = árvore 🌳

**main** = tronco principal (versão oficial)

**Branches** = galhos (experimentos, novas funcionalidades, correções)

**Merge** = colar o galho de volta no tronco

👉 Agora, vou organizar esse passo a passo como se fosse um guia prático de uso do Git + GitHub.

Vou explicar cada situação com os comandos corretos e observações importantes.

### ◆ Situação 1: Configurar o ambiente antes de enviar os arquivos

Ou seja, preparar sua pasta local para ser um repositório Git.

# Entrar na pasta do projeto

**cd nome-da-pasta**

# Inicializar o repositório Git na pasta

**git init**

# Configurar seu usuário (só precisa fazer uma vez no PC)

**git config --global user.name "Seu Nome"**

**git config --global user.email "seuemail@example.com"**

# Criar um README (opcional, mas recomendado)

**echo "# Meu Projeto" >> README.md**

# Adicionar todos os arquivos

**git add .**

# Fazer o primeiro commit

**git commit -m "Primeiro commit: configuração inicial"**

## ◆ Situação 2: Enviar o projeto local para o GitHub remoto

No GitHub, crie um novo repositório (sem README, se você já criou um local).

Depois copie a URL do repositório (HTTPS ou SSH). Exemplo (HTTPS):

# Adicionar o repositório remoto

**git remote add origin <https://github.com/seu-usuario/nome-do-repo.git>**

# Enviar os arquivos locais para o remoto (GitHub)

**git branch -M main # garante que está usando a branch "main"**

**git push -u origin main**

Agora seu projeto local está sincronizado com o remoto.

### ◆ Situação 3: Atualizar o projeto remoto depois de mudanças locais

Sempre que você modificar arquivos locais:

# Ver quais arquivos foram modificados

**git status**

# Adicionar arquivos modificados

**git add .**

# Fazer commit das mudanças

**git commit -m “Descrição das alterações feitas”**

# Enviar para o GitHub

**git push origin main**

#### ◆ Situação 4: Trazer um arquivo do GitHub remoto que não existe localmente

Aqui depende:

Se você já tem o repositório clonado, basta atualizar:

**git pull origin main**

Isso vai trazer o arquivo que existe no remoto mas não no local.

Se você não tem o repositório clonado ainda, faça:

**git clone https://github.com/seu-usuario/nome-do-repo.git**

## ◆ Situação 5: Listar os repositórios que você tem no GitHub remoto via terminal

⚠ **ATENÇÃO:** O Git sozinho não consegue listar repositórios do GitHub, porque ele só controla repositórios locais + conexões configuradas.

Para listar seus repositórios do GitHub sem abrir o navegador, você precisa usar a GitHub CLI (gh).

Exemplo (com a CLI instalada e autenticada):

# Fazer login (uma vez só)

**gh auth login**

# Listar repositórios do usuário

**gh repo list seu-usuario --limit 100**

Se você quiser fazer isso apenas com Git “puro”, não tem jeito — o Git não conversa diretamente com a API do GitHub.

## ◆ Situação 6: Alterar (deletar, renomear, etc.) repositórios do GitHub remoto via terminal

De novo, com o Git puro isso não é possível. O Git apenas envia/recebe commits e branches.

Para administrar repositórios (criar, deletar, renomear, alterar permissões, etc.) você precisa da GitHub CLI.

Exemplos com GitHub CLI:

# Deletar repositório

**gh repo delete seu-usuario/nome-do-repo**

# Renomear repositório

**gh repo rename novo-nome**

# Criar um repositório novo

**gh repo create novo-repo --public --source=. --remote=origin --push**

### ✅ Resumo importante:

**Git** = controla versões de arquivos local + envia/recebe commits de repositórios remotos.

**GitHub** = plataforma de hospedagem.

**Para gerenciar repositórios remotos sem navegador** → usar GitHub CLI (gh).

As vezes precisamos configurar o user.mail e user.nome localmente e globalmente. Isso acontece bastante quando várias pessoas usam o mesmo computador, mas cada uma tem seu GitHub.

### ◆ Como funciona o Git com nome/e-mail

O comando que você já usou:

```
git config --global user.name "Seu Nome"
```

```
git config --global user.email "seuemail@example.com"
```

Esse **--global** significa que a configuração vale para o computador inteiro (todos os projetos).

**Mas... você pode sobrescrever isso em cada projeto usando:**

```
git config --local user.name "Nome do Colega"
```

```
git config --local user.email "emaildo.colega@example.com"
```

👉 Assim, dentro da pasta do projeto do seu colega, o Git vai usar o nome e o e-mail dele, sem alterar sua configuração global.

## ◆ Passo a passo para seu colega usar seu computador

**Ele entra na pasta do projeto dele:**

```
cd projeto-do-colega
```

**Configura o Git só para esse repositório:**

```
git config user.name "Nome do Colega"
```

```
git config user.email "emaildo.colega@example.com"
```

**Inicia o repositório e faz os commits:**

```
git init
```

```
git add .
```

```
git commit -m "Primeiro commit do colega"
```

**Adiciona o repositório remoto dele (no GitHub dele, não no seu):**

```
git remote add origin https://github.com/colega/repositorio.git
```

```
git branch -M main
```

```
git push -u origin main
```

## ◆ Sobre autenticação (senha/token)

Mesmo que ele configure nome/e-mail, ainda vai precisar autenticar no GitHub dele para enviar (git push).

Hoje em dia, o GitHub não aceita senha normal, só token pessoal ou SSH Key.

Então:

Se usar HTTPS → ele digita o token dele quando o Git pedir.

Se usar SSH → ele precisa configurar uma chave SSH própria no GitHub dele.

## 📌 Resumindo:

O **--global** é para todo o PC.

O **--local** (ou simplesmente sem --global) é só para o projeto atual.

Então, mesmo que o seu PC esteja configurado com seu nome e seu e-mail, seu colega pode configurar os dele apenas na pasta do projeto dele, sem afetar você.